FlatSat Mission Control

Software Manual

Ground-station dashboard for the FlatSat Satellite Learning Kit

Application	flatsat_dashboard 1.0.0+1
Covers	Dashboard · PC Bridge · OBC / COMMS / GS firmware
Audience	Students, operators, instructors, developers
Reference	flatsat.kidorbit.space

Contents

PART I

User Guide

1. Introduction

FlatSat Mission Control is the ground-station software for the FlatSat Satellite Learning Kit. It is the desktop application an operator uses to watch live telemetry from the satellite, send telecommands to it, and retrieve images the camera payload has captured.

The FlatSat kit itself is an educational platform that lays the core components of a CubeSat out on a flat, accessible plane so they can be studied, programmed and tested. The kit is split into two segments: the Space Segment (the FlatSat board — OBC, EPS, COMMS and camera payload) and the Ground Segment (a separate STM32-based radio board that plugs into a computer). Mission Control is the software layer that sits on top of the Ground Segment.

1.1 The signal chain

Everything you see in the dashboard travels through the same chain. Understanding it makes troubleshooting far easier, because each link fails in a recognisably different way.

```
Dashboard (Flutter app)
        |  WebSocket  ws://localhost:8080  (JSON)
PC Bridge (gs_bridge.py)
        |  USB serial  115200 baud  (KISS frames)
Ground Station board  (STM32F103RC + RFM98PW)
        )))  433 MHz radio link  (((
COMMS  (STM32F411RE + RFM98PW)
        |  UART
OBC  (STM32F429ZI)  --I2C-->  EPS  |  GPS  |  Camera
```

The two halves matter separately. The USB half connects the app to the ground station on your desk. The radio half connects the ground station to the satellite. The dashboard reports on each half independently, so a red banner always tells you which one is at fault.

1.2 What you can do with it

- Watch live telemetry — battery percentage, voltage, temperature, subsystem power states and error flags, refreshed from a beacon that the OBC transmits roughly every 10 seconds.
- Send telecommands — ping the satellite, request a status scan, ask for a GPS fix, pull an EPS reading, or switch subsystem power channels on and off.
- Operate the camera payload — trigger a capture at a chosen resolution, list the images stored on the satellite, download them over the radio link, and delete them.
- Monitor the radio link — RSSI and SNR are reported per packet, and the dashboard grades the link as Stable, Not stable, or No signal.
- Log power data — record EPS readings on the PC over time, or configure the satellite itself to log at a fixed interval and pull the log down later.

2. Before you begin

The one-click launcher handles almost everything, but two prerequisites must be in place before it can work.

Requirement	Detail
Python 3	Required, with pip. The launcher uses it to install and run the bridge. On Windows and macOS install it from python.org; on Linux and Raspberry Pi it is normally preinstalled.
Flutter	Only required if you run the dashboard from source. If the app has been built once ahead of time, the launcher uses the built binary and Flutter is not needed.
Hardware	The Ground Station board connected to the computer by USB, plus the FlatSat powered on with antennas attached on both the OBC/COMMS side and the ground station side.

The bridge itself needs only two Python packages, pyserial and websockets. The launcher installs them for you on first run.

2.1 Platform-specific setup

The dashboard ships a built-in setup guide covering the same ground (Settings → Open setup guide). It is reproduced here for reference.

Windows

1. Install Python 3 from python.org — tick "Add python.exe to PATH" during setup.
2. If the ground-station COM port does not appear, install its USB-serial driver (CP210x, CH340 or FTDI).
3. Close the Arduino IDE Serial Monitor before running — it locks the port.
4. Start everything by double-clicking run_mission_control.bat.
5. If auto-detect picks the wrong port, choose the GS port in the dashboard using Change port.

macOS

1. Install Python 3 (from python.org, or with brew install python).
2. Install a USB-serial driver (CH340 or CP210x) if your adapter is not recognised.
3. On first launch, right-click run_mission_control.command and choose Open, to get past Gatekeeper. After that a double-click works.
4. Close the Arduino Serial Monitor before running.
5. Pick the GS port in the dashboard if needed (Change port).

Linux and Raspberry Pi

1. Python 3 is usually preinstalled; the launcher installs pyserial and websockets for you.
2. Grant yourself serial access: sudo usermod -aG dialout $USER — then log out and back in.
3. Close the Arduino Serial Monitor before running.
4. Start with ./run_mission_control.sh.
5. The GS KISS link is a USB-serial adapter (usually /dev/ttyUSB0) — pick it in the port selector if auto-detection misses it.

3. Installation and first run

Plug the ground station into the computer with a USB cable, then run the launcher for your operating system.

Operating system	Double-click this file
Windows	`run_mission_control.bat`
macOS	`run_mission_control.command`
Linux / Raspberry Pi	`run_mission_control.sh`

The launcher performs four steps automatically:

1. Installs the Python dependencies (first run only).
2. Stops any leftover bridge process and frees TCP port 8080, then starts a fresh ground-station bridge.
3. Opens the dashboard — preferring a prebuilt binary, and falling back to flutter run if none exists.
4. Stops the bridge again when you close the app.

There are no ports or settings to configure by hand. The bridge finds the ground station itself.

> **macOS first run:** right-click run_mission_control.command and choose Open once, to get past Gatekeeper. A double-click works from then on.

3.1 Preparing computers for a classroom

Building the app ahead of time means the launcher runs a ready-made binary, so students' computers do not need Flutter installed — only Python. From the Dashboard folder:

```
flutter pub get
flutter build linux      # or:  flutter build windows  /  flutter build macos
```

The launcher automatically prefers the built app when it finds one.

4. Using the dashboard

4.1 The connection banner

A banner across the top of the app always states exactly where the connection stands. Read it first whenever something is not working.

Banner message	What it means and what to do
Starting ground station software…	The bridge is still coming up. Wait a few seconds.
No ground station detected on USB	The USB half of the chain is broken. Plug in the ground station, or use the port dropdown to pick it and press Reconnect. Close the Arduino Serial Monitor if it is open — it locks the port.
Waiting for satellite…	USB is fine; the radio half is not. Power on the OBC and COMMS boards and check that both antennas are attached. Beacons arrive roughly every 10 seconds.
Live	Telemetry is flowing normally.

Two status chips beside the banner summarise the same thing at a glance: BRIDGE / OFFLINE for the USB side, and SAT LINK / NO LINK for the radio side.

4.2 Telemetry cards

The top row of the dashboard shows the four headline values carried in every beacon.

Card	Shows
Battery	State of charge as a percentage, from the EPS.
Temperature	Board temperature in degrees Celsius, read from the TMP102 sensors.
Signal	RSSI in dBm, and SNR where the radio mode provides it.
Link status	A three-state grade: Stable, Not stable, or No signal.

The link grade is worth understanding. "No signal" means nothing is being received at all. "Stable" means packets are arriving and RSSI is at or above −100 dBm. "Not stable" means packets are still arriving, but the signal has dropped below that threshold and frames will start to be lost. SNR is a LoRa-only measurement — in FSK mode the ground station reports a sentinel value and the dashboard hides the field rather than showing a misleading number.

4.3 Commands

Commands are issued from the button row. Each one sends a telecommand up the radio link and waits for the satellite to answer.

Button	Effect
PING	Sends a ping and waits for an acknowledgement. The quickest way to confirm the radio link is alive in both directions.
BEACON	Requests a beacon frame — the standard telemetry packet — rather than waiting for the next scheduled one.
SCAN	Runs a health scan and reports which I2C devices the OBC can currently see.
GET GPS	Asks the OBC for a position fix: latitude, longitude, altitude and satellite count. If the receiver has no fix yet the dashboard says so explicitly rather than showing zeros.
GET EPS	Pulls a full power-system reading — bus voltage and current from the INA226 monitors, temperatures from the TMP102 sensors, and output voltage from the ADM1177 switches.
TAKE PIC	Opens the capture dialog, then commands the camera payload to take a photograph.
LIST	Lists the image files currently stored on the satellite.

Subsystem power channels are switched from the SUBSYSTEM POWER panel rather than the button row. Each card shows the channel's present state and toggles it when tapped.

Channel	Notes
Communication	Protected by a safety lock in Settings, because switching COMMS off from the ground severs the link that would let you switch it back on.
Payload 1 / GPS	GPS receiver power.
Payload 2 / PC104	Power for hardware attached to the PC104 expansion slots.
Camera	Only shown in Production camera mode. In Prototype mode the camera is permanently powered and no card appears.

4.4 The EPS panel

The EPS section presents the satellite's power telemetry grouped by sensor family, with each reading labelled by its device type and I2C address so the display maps directly onto the hardware:

- INA226 — bus voltage and current for each monitored rail.
- TMP102 — temperature at each sensed location.
- ADM1177 — output voltage on each switched power channel.

The panel can also plot history over time. What history is available depends on the EPS logging mode you choose in Settings (see section 5).

4.5 Working with images

Imaging is the FlatSat's mission payload, and the full cycle can be flown from the dashboard.

1. Press TAKE PIC and choose a resolution. The dialog explains the trade-off directly: a higher resolution gives a sharper picture but a bigger file and a slower download over the radio.
2. Press LIST to see what is stored on the satellite. Each entry shows its file size.
3. Press the download icon beside a file to bring it down. A progress bar appears, and the transfer can be paused and resumed.
4. When the download completes, a "Show in folder" button opens the saved image on your computer.
5. Press the delete icon to remove a file from the satellite and free storage.

Setting	Resolution	Character
VGA	640 × 480	Smallest file, fastest download. The default.
SVGA	800 × 600	A modest step up.
UXGA	1600 × 1200	Middle ground.
3 MP	2048 × 1536	Large.
5 MP	2592 × 1944	Full sensor resolution; slowest to transfer.

> **Bear in mind:** images cross the radio link in 32-byte chunks at a few kilobits per second. A full 5 MP capture takes a long time to come down. VGA is the sensible choice for routine work.

4.6 Appearance

The sun and moon button at the top right switches between the light and dark themes. Two optional terminal panels — an event log and a raw bridge-traffic view — can be shown or hidden from Settings.

5. Settings reference

Settings are reached from the gear icon at the top right. Changes persist across restarts.

Section	Options
Camera mode	Prototype or Production. This must match the firmware flashed on the OBC. In Prototype the camera is always powered and no camera power card is shown; in Production the camera is switched on PD4 and the card appears.
Panels	Show or hide the event log terminal and the bridge traffic terminal.
Capture	The default capture resolution, remembered between sessions.
Display	Whether image sizes are shown in KB only, or in both KB and MB.
EPS logging	Three modes — Off (live readings only); Live + keep history on PC (the dashboard accumulates readings locally); and Log on satellite (the OBC records at a set interval and you pull the log down later). The interval is adjustable from 1 to 3600 seconds, defaulting to 5.
Image save folder	Where downloaded images are written. Defaults to your Documents folder.
Safety	Lock Communication power OFF. Enabled by default; it prevents switching the COMMS channel off from the ground, which would cut the link irrecoverably. Unlocking requires an explicit confirmation.
Satellite clock	Sync time now — pushes the computer's clock to the satellite so that telemetry and stored files carry meaningful timestamps.
Help	Opens the built-in setup guide.

6. Troubleshooting

Symptom	Resolution
Link status stays red, or "No ground station detected"	Close the Arduino IDE Serial Monitor — it holds the USB port open. Confirm the ground station is plugged in and powered. Open the port dropdown in the banner and pick the device manually. STM32 boards usually appear as ttyACM0 on Linux, usbmodem… on macOS, or a COM… port on Windows; a USB-UART adapter appears as ttyUSB0.
"Permission denied" on the serial port (Linux / Raspberry Pi)	Run: sudo usermod -a -G dialout $USER — then log out and back in.
Banner shows "Waiting for satellite…" and never goes Live	The USB side is fine; the radio side is not. Power the OBC and COMMS boards and attach both antennas. The OBC beacons about every 10 seconds. Confirm that frequency, sync word and bitrate match between the COMMS module and the ground station.
Auto-detect picks the wrong port	OBC and COMMS debug consoles also enumerate as ttyACM devices, so the name alone is ambiguous. The bridge probes each candidate for real KISS traffic, but if it still guesses wrong, select the correct port manually and press Reconnect.
Downloads stall or fail	Check the link grade. A "Not stable" link loses chunks; the bridge retries each chunk up to five times before giving up. Move the antennas or reduce the capture resolution.
Need to see detailed logs	The bridge writes everything to PC_Bridge/bridge_log.txt. The bridge traffic terminal in the app shows the same stream live.

PART II

Technical Reference

7. Architecture

Mission Control is deliberately split into a thin transport layer and a presentation layer. The bridge owns everything that touches hardware — the serial port, KISS framing, CRC verification, retries and the image-transfer state machine — and exposes a single JSON WebSocket. The dashboard owns nothing but presentation and user intent. This separation is what allows the app to be rebuilt, replaced or run on a different machine without touching protocol code.

Component	Technology	Responsibility
Dashboard	Flutter / Dart	Desktop UI. Renders telemetry, issues commands, manages downloaded images and user settings. Runs on Windows, macOS and Linux.
PC Bridge	Python 3	Translates between KISS-framed serial traffic and JSON over WebSocket. Handles port detection, CRC32 verification, link-timeout detection, retries and resumable image download.
GS firmware	STM32F103RC	Radio modem. Modulates and demodulates 433 MHz packets and injects RSSI/SNR metadata for received frames.
COMMS firmware	STM32F411RE	Satellite-side radio. Passes ground packets to the OBC over UART and transmits the OBC's telemetry back down.
OBC firmware	STM32F429ZI	Satellite brain. Executes telecommands, drives the EPS over I2C, reads GPS, operates the camera, and manages SD-card storage.

7.1 Data flow

A telecommand travels down the chain and its response comes back up the same path:

- The user presses a button. The dashboard sends a small JSON object over the WebSocket, for example {"cmd":"get_gps"}.
- The bridge maps that to a numeric opcode, builds an application packet with a CRC32, wraps it in a KISS frame, and writes it to the serial port.
- The GS board transmits the frame at 433 MHz. COMMS receives it and forwards the payload to the OBC over UART.
- The OBC executes the command and replies. The response returns through COMMS and the GS board, which appends RSSI and SNR metadata.
- The bridge decodes the KISS frame, verifies the CRC32, updates its telemetry state, and broadcasts a typed JSON message to every connected WebSocket client.
- The dashboard receives the message and re-renders the affected panel.

8. The PC bridge

The bridge (PC_Bridge/gs_bridge.py) is a single Python process. It opens the serial port in a background thread and serves a WebSocket on localhost.

Parameter	Value
WebSocket endpoint	`ws://localhost:8080`
Serial baud rate	`115200`
Link timeout	Fifteen seconds without a packet before the link is declared LOST
Image chunk size	32 bytes
Chunk timeout / retries	2.5 s per chunk, up to 5 retries
Download ceiling	1800 s for a single transfer
Default download folder	`./downloads`
Log file	`PC_Bridge/bridge_log.txt`

8.1 Serial port detection

Port selection resolves in priority order:

1. A command-line argument — python3 gs_bridge.py /dev/ttyACM0 (or COM4, and so on).
2. The FLATSAT_SERIAL_PORT environment variable.
3. Automatic detection.

Auto-detection is more careful than a simple name match, because it has to be. Bluetooth ports are skipped, since opening them can hang on Windows. Legacy motherboard UARTs (/dev/ttyS0–31) and the Raspberry Pi's own UART (/dev/ttyAMA*) are excluded — they are never the ground station, always fail to open, and would add around twenty seconds to the probe. Remaining candidates are ranked, with STM32 native-USB CDC ACM devices first and USB-UART adapters (FTDI, CP210x, CH340) second.

Ranking alone is still not conclusive, because the OBC and COMMS debug consoles also enumerate as ttyACM devices while the GS KISS link may be a ttyUSB adapter. So when more than one candidate survives, the bridge opens each in turn, sends a ping, and listens for a valid KISS frame containing the AA BB sync pattern — a quick pass first, then a longer pass covering the full 10-second beacon interval. The first port that actually talks is the one it keeps. If none does, it falls back to the highest-ranked candidate and logs a warning suggesting manual selection.

8.2 Reliability behaviour

- Every application packet carries a CRC32, verified on receipt.
- Multi-part responses (image lists, EPS logs) are reassembled by a chunk collector that re-requests the command if no new chunk arrives within 1.3 seconds, up to 8 retries.
- A duplicate guard swallows the redundant re-send that can follow a completed transfer, and stale reassemblies are abandoned after 4 seconds.
- Image downloads are resumable, and can be paused and resumed from the dashboard.
- GPS requests retry up to 5 times, and EPS log requests up to 10, before reporting failure to the app.

9. WebSocket API

The dashboard and the bridge exchange JSON objects over ws://localhost:8080. Because the interface is plain JSON on a local socket, any client can drive the satellite — a Python script, a test harness, or a replacement UI.

9.1 Client to bridge

Every message from the client carries a "cmd" key.

cmd	Parameters	Action
`ping`	—	Ping the satellite.
`status`	—	Request a health scan of I2C devices.
`beacon`	—	Request a beacon telemetry frame.
`get_gps`	—	Request a GPS fix.
`get_eps`	—	Request a full EPS reading.
`take_pic`	`resolution`	Capture an image (resolution 0–4, VGA to 5 MP).
`toggle_pwr`	`subsystem`	Toggle a power channel: 0 Communication, 1 Payload 1 / GPS, 2 Payload 2 / PC104, 3 Camera.
`list_image`	—	List stored images.
`download`	`filename`	Start a resumable image download.
`remove_image`	`filename`	Delete an image from the satellite.
`pause_download`	—	Pause the transfer in progress.
`resume_download`	—	Resume a paused transfer.
`sync_time`	—	Push the PC clock to the satellite.
`eps_log_config`	`enable, interval`	Configure on-satellite EPS logging.
`get_eps_log`	—	Retrieve the stored EPS log.
`list_ports`	—	List available serial ports.
`set_port`	`port`	Select a serial port explicitly.
`reconnect`	—	Reopen the serial port.
`set_download_dir`	`path`	Change where downloaded images are written.

Commands that would collide with an active image download are rejected with a busy message rather than being queued.

9.2 Bridge to client

Every message from the bridge carries a "type" key and a "data" payload.

type	Payload
telemetry	The current telemetry snapshot (see section 11).
bridge_status	serial_connected, port, available_ports, error — this drives the connection banner and port picker.
ack / nack	Command acknowledged or refused.
gps / gps_failed	A position fix, or a failure after retries are exhausted.
eps / eps_failed	A power-system reading, or a failure.
health	The list of I2C devices discovered by a scan.
image_list	Files stored on the satellite. Large lists also emit image_list_progress and image_list_failed.
eps_log	Stored EPS log records, with eps_log_progress and eps_log_failed during retrieval.
download_started	A transfer has begun.
download_progress	Chunk-level progress.
download_paused / download_resumed	Transfer state changes.
download_complete	The transfer finished; includes the saved path.
download_failed	The transfer failed.
busy	The command was rejected because a download is in progress.
error	Malformed or unknown command.
bridge_log	A raw log line, shown in the bridge traffic terminal.

10. Radio link and packet protocol

10.1 Link parameters

Both ends of the link must be configured identically. A mismatch in frequency, sync word or bitrate produces exactly the same symptom as a dead radio: the banner sits at "Waiting for satellite…" indefinitely.

Parameter	Ground Station	COMMS (satellite)
MCU	STM32F103RC (Cortex-M3)	STM32F411RE
Radio module	RFM98PW	RFM98PW
Frequency	433 MHz	433 MHz
Transmit power	27 dBm	27 dBm
Data rate	9.6 kbps	4.8 – 9.6 kbps
PC connection	USB-C (UART via ST-Link)	—
Antenna	SMA connector	Integrated, plus external connector

10.2 KISS framing

Packets are framed using KISS, the standard amateur-radio framing protocol. A frame is delimited by FEND bytes; any FEND or FESC occurring inside the payload is escaped so it cannot be mistaken for a delimiter.

Symbol	Value	Meaning
FEND	`0xC0`	Frame delimiter.
FESC	`0xDB`	Escape character.
TFEND	`0xDC`	Transposed frame end — follows FESC to represent a literal 0xC0.
TFESC	`0xDD`	Transposed frame escape — follows FESC to represent a literal 0xDB.

Inside the frame, the application packet begins with an AA BB sync pattern, carries a command opcode and payload, and ends with a CRC32 over the packet contents.

10.3 Command opcodes

These are the numeric opcodes carried on the air. They are useful when writing firmware, debugging with a serial monitor, or building an alternative client.

Opcode	Name	Direction and meaning
0x01	PING	Ground → satellite. Liveness check.
0x02	ACK	Satellite → ground. Command accepted.
0x03	BEACON	Periodic telemetry frame, also requestable.
0x04	TAKE_PIC	Capture an image at the given resolution.
0x05	GET_GPS	Request a position fix.
0x06	TOGGLE_PWR	Switch a power channel.
0x07	LIST_IMAGE	Request the stored image list.
0x08	REMOVE_IMAGE	Delete a stored image.
0x09	REQ_CHUNK	Request a specific chunk during download.
0x0A	STATUS	Request an I2C health scan.
0x0B	IMAGE_DATA	Satellite → ground. One 32-byte image chunk.
0x0C	GPS_DATA	Satellite → ground. Position fix.
0x0D	IMAGE_LIST	Satellite → ground. Image list.
0x0E	NACK	Satellite → ground. Command refused or failed.
0x0F	GET_EPS	Request a power-system reading.
0x10	EPS_DATA	Satellite → ground. Power-system reading.
0x11	HEALTH_DATA	Satellite → ground. Health-scan result.
0x12	SET_TIME	Set the satellite clock.
0x13	EPS_LOG_CFG	Configure on-satellite EPS logging.
0x14	GET_EPS_LOG	Request the stored EPS log.
0x15	EPS_LOG_DATA	Satellite → ground. EPS log records.
0x16	EPS_LOG_CLEAR	Clear the stored EPS log.
0x17	IMAGE_LIST_DATA	Satellite → ground. Image list as a uint16-chunked file.

11. Telemetry data model

The beacon carries a compact fixed set of fields, which the bridge decodes and republishes as JSON.

JSON field	Type	Meaning
system_errors	int	Bitmask of subsystem faults (see below).
payload_pwr	bool	Communication channel power state.
gps_pwr	bool	Payload 1 / GPS channel power state.
cam_pwr	bool	Payload 2 / PC104 channel power state.
battery_pct	int	Battery state of charge, percent.
temperature	int	Board temperature, degrees Celsius.
voltage	int	Battery bus voltage.
link_status	string	ACTIVE or LOST.
rssi	double	Received signal strength, dBm, injected by the GS board.
snr	double	Signal-to-noise ratio. LoRa only; −12.8 is the FSK "not applicable" sentinel.
last_packet_time	double	Timestamp of the most recent packet, used for timeout detection.

11.1 System error bitmask

system_errors is a bit field. Zero means no errors; otherwise the dashboard renders the set bits as a comma-separated list.

Bit	Label	Fault
0x01	I2C	I2C bus fault.
0x02	SD	SD card fault.
0x04	EPS	Electrical power system fault.
0x08	CAM	Camera payload fault.
0x10	GPS	GPS receiver fault.

11.2 GPS data

A GPS response carries latitude, longitude, altitude, satellite count, and a fix-valid flag. The flag matters: it distinguishes "the radio link worked but the receiver has no fix" from "the request failed", and the dashboard renders those two cases differently instead of showing zeros.

12. Hardware context

Mission Control is only meaningful alongside the hardware it commands. The kit's architecture centres on a main board where subsystems are integrated over a standardised PC104 bus. A notable design choice is that the EPS has no microcontroller of its own — its sensors and switches hang directly off the OBC's I2C bus, so all power control is executed by OBC firmware.

Subsystem	Core MCU	Role
OBC	STM32F429ZI	Mission logic and data handling; also the controller for the EPS hardware. Supports an external SD card and 2 MB of internal flash.
EPS	None — driven by OBC	Li-ion 18650 battery (17 Wh+), solar charging simulation, and a network of I2C sensors and switches: INA226 current/voltage monitors, TMP102 temperature sensors, ADM1177 power switches.
COMMS	STM32F411RE	RFM98PW radio at 433 MHz. KISS framing, packet handling, and a UART link to the OBC.
Payload	Camera module	5 megapixel imager; transfers image data to the OBC over SPI.
Ground Segment	STM32F103RC	RFM98PW radio at 433 MHz with a USB-C interface to the operator's computer.

Two PC104 expansion slots allow custom subsystems to be added. Each provides two controllable power channels plus direct UART and CAN connections to the OBC — which is why Payload 2 / PC104 appears as a switchable channel in the dashboard.

13. Building and running from source

To run the two halves manually rather than through the launcher:

```
# 1) Start the bridge (auto-detects the port; override if needed)
cd PC_Bridge
pip install -r requirements.txt
python3 gs_bridge.py                 # or: python3 gs_bridge.py /dev/ttyACM0
#   or:  FLATSAT_SERIAL_PORT=/dev/ttyACM0 python3 gs_bridge.py

# 2) Start the app
cd ../Dashboard
flutter run
```

The dashboard targets Dart SDK 3.5 or later. Its principal dependencies are web_socket_channel for bridge communication, fl_chart for the EPS plots, provider for state management, window_manager for the custom frameless title bar, shared_preferences for persisted settings, and file_picker with path_provider for the image save folder.

> **Note:** if you start the bridge by hand while the launcher is also running, the second instance will fail to bind port 8080 and the app will end up talking to a bridge with no serial data. Stop one before starting the other.

14. Project layout

```
FlatSat_New_Architecture/
├── run_mission_control.sh        # Linux / Raspberry Pi launcher
├── run_mission_control.command   # macOS launcher
├── run_mission_control.bat       # Windows launcher
├── install_linux_icon.sh         # Installs the Linux menu entry
├── Dashboard/                    # Flutter dashboard app
│   └── lib/
│       ├── main.dart
│       ├── models/               # TelemetryData, GpsData
│       ├── screens/              # dashboard_screen.dart
│       ├── services/             # websocket, settings, bridge_launcher
│       ├── theme/                # Light and dark themes
│       ├── utils/                # EPS history, OS file-open helpers
│       └── widgets/              # Banner, EPS panel, dialogs, cards
├── PC_Bridge/
│   ├── gs_bridge.py              # Serial (KISS) <-> WebSocket bridge
│   ├── test_replay.py            # Offline replay harness
│   ├── requirements.txt          # pyserial, websockets
│   └── bridge_log.txt            # Written at runtime
├── OBC/    OBC.ino               # On-Board Computer firmware
├── COMMU/  COMMU.ino             # Communications firmware
└── GS/     GS.ino                # Ground Station firmware
```

15. Further reading

The official FlatSat hardware documentation, including per-subsystem block diagrams, pinouts and example firmware, is published at flatsat.kidorbit.space.

Page	Address
Overview	https://flatsat.kidorbit.space/
On-Board Computer	https://flatsat.kidorbit.space/subsystems/obc/
Electrical Power System	https://flatsat.kidorbit.space/subsystems/eps/
Communication System	https://flatsat.kidorbit.space/subsystems/communication/
Payload	https://flatsat.kidorbit.space/subsystems/payload/
Ground Station	https://flatsat.kidorbit.space/subsystems/ground-segment/

The COMMS and GS example sketches depend on the RadioLib Arduino library; the published code targets version 7.5.0, installable from the Arduino IDE Library Manager.